Vilnius University

MINI PROJECT

INTERNATIONAL CYBERSECURITY AND CYBERINTELLIGENCE

2026



VILNIUS UNIVERSITY
FACULTY OF MATHEMATICS AND INFORMATICS
INSTITUTE OF COMPUTER SCIENCE
CYBERSECURITY LABORATORY

International Cybersecurity and Cyberintelligence 1st year Mini Project

Supply Chain Security

Tiekimo Grandinės Saugumas

Done by:
Ariana Alves
Greivin Sanchez Garita
Juan Luis Ramos

Supervisor:
prof. dr. Agnė Brilingaitė

Vilnius
2026

Contents

Abstract **4**

Santrauka **5**

1 Introduction **6**

2 Related Work **7**

3 Requirements **8**
3.1 Functional Requirements 8
3.2 Non-Functional Requirements 9

4 Methodology **9**
4.1 System Architecture 9
4.2 Risk Scoring Heuristic 12
4.2.1 CVE Contribution 12
4.2.2 Inactivity Contribution 13
4.2.3 Bus Factor Contribution 13
4.2.4 Issue Neglect Contribution 13
4.2.5 Zombie Dependency Classification 14
4.3 Overall Repository Security Heuristic 14
4.4 Repository Security Classification 15

5 Case Analysis **15**
5.1 Heuristic Refinement and "Zombie Package" Detection 16

6 Conclusions **17**

References **18**

Abstract

Modern software development heavily depends on open-source libraries and external package ecosystems, significantly increasing exposure to software supply chain attacks. Traditional dependency scanners primarily focus on publicly disclosed vulnerabilities, often overlooking long-term risks associated with abandoned or poorly maintained dependencies. This project presents a lightweight Software Supply Chain Security framework capable of analyzing public repositories and evaluating dependency health through a combination of vulnerability intelligence and repository vitality metrics.

The proposed system extracts dependencies from common package manifest files, integrates vulnerability information from the Open Source Vulnerability (OSV) database, and collects maintenance-related metadata from GitHub repositories, including commit activity, contributor diversity, and unresolved issue accumulation. These indicators are combined through a heuristic-based mathematical model that generates a dependency risk score ranging from 0 to 100 and classifies high-risk abandoned dependencies as Zombie Packages.

To improve accessibility and operational usability, the framework was implemented as both a command-line utility and a web-based platform capable of generating structured JSON audit reports and real-time visual risk summaries. Experimental evaluation using real-world repositories demonstrated that the proposed approach can identify latent supply chain risks that remain undetected by conventional CVE-centric analysis methods.

The results indicate that combining vulnerability analysis with repository sustainability metrics provides a more proactive approach for identifying insecure dependencies and improving software supply chain risk assessment.

Santrauka

Šiuolaikinė programinės įrangos kūrimo kultūra yra stipriai priklausoma nuo atvirojo kodo bibliotekų ir išorinių paketų ekosistemų, o tai reikšmingai padidina programinės įrangos tiekimo grandinės atakų riziką. Tradiciniai priklausomybių analizatoriai daugiausia dėmesio skiria viešai paskelbtoms pažeidžiamumų bazėms, dažnai neįvertindami ilgalaikės rizikos, susijusios su apleistomis ar prastai prižiūrimomis priklausomybėmis. Šiame projekte pristatoma lengvasvorė programinės įrangos tiekimo grandinės saugumo sistema, galinti analizuoti viešąsias saugyklas ir vertinti priklausomybių būklę, derinant pažeidžiamumų žvalgybą su saugyklų gyvybingumo rodikliais.

Siūloma sistema išskiria priklausomybes iš įprastų paketų konfigūracijos failų, integruoja informaciją apie pažeidžiamumus iš „Open Source Vulnerability“ (OSV) duomenų bazės ir renka priežiūros metaduomenis iš „GitHub“ saugyklų, įskaitant programinio kodo atnaujinimo (angl. commit) veiklą, bendraautorių įvairovę ir neišspręstų problemų (angl. issues) kaupimąsi. Šie rodikliai derinami pasitelkiant heuristinį matematinį modelį, kuris sugeneruoja priklausomybės rizikos balą nuo 0 iki 100 ir klasifikuoja didelės rizikos apleistas priklausomybes kaip „paketus zombius“ (angl. Zombie Packages).

Siekiant pagerinti prieinamumą ir operacinį patogumą, sistema buvo įgyvendinta tiek kaip komandinės eilutės įrankis, tiek kaip internetinė platforma, galinti generuoti struktūrizuotas JSON audito ataskaitas bei realaus laiko vizualines rizikos santraukas. Eksperimentinis vertinimas naudojant realaus pasaulio saugyklas parodė, kad siūlomas požiūris gali nustatyti paslėptas tiekimo grandinės rizikas, kurios lieka nepastebėtos taikant įprastus, tik į CVE orientuotus analizės metodus.

Rezultatai rodo, kad pažeidžiamumų analizės derinimas su saugyklų tvarumo rodikliais užtikrina proaktyvesnį nesaugių priklausomybių identifikavimą ir pagerina programinės įrangos tiekimo grandinės rizikos vertinimą.

1. Introduction

Modern software development is increasingly dependent on open-source libraries and external contributors, creating a complex supply chain that introduces critical security blind spots. Although traditional security measures often prioritize the primary codebase, attackers have systematically shifted their focus toward third-party dependencies and contributor credentials to inject malicious code upstream. Because developers frequently integrate packages from public repositories such as PyPI to optimize development velocity, they inadvertently import hidden risks. Once a package is installed, the system implicitly trusts it, meaning any security flaw or malicious script nested within a sub-dependency is propagated directly into the final software product. This blind trust is failing the industry; the 2025 Verizon Data Breach Investigations Report (DBIR) revealed that 30% of all data breaches involve a third-party compromise, representing a 100% increase from previous years [15].

The danger of this reliance is exemplified by the supply chain attack on LiteLLM in March 2026. A threat group known as TeamPCP exploited a vulnerability in a third-party CI/CD tool to steal the publishing credentials for LiteLLM, subsequently uploading maliciously modified versions (v1.82.7 and v1.82.8) to the Python Package Index [12]. When automated deployment pipelines downloaded these updates, a hidden, multi-stage credential stealer was executed. The malware performed active reconnaissance for cloud architecture configurations, Kubernetes tokens, and database passwords, resulting in catastrophic data leaks [4]. Furthermore, the impact of such compromises extends far beyond server environments. When a compromised Python dependency infects a CI/CD pipeline, the malware can tamper with downstream products during the build process. For instance, if the infected environment is utilized to compile mobile applications, the script can quietly alter the source code to introduce severe structural weaknesses, such as pending intent hijacking or direct data leaks, directly into the final application distributed to end-users. Such build-time tampering underscores the nature of contemporary supply chain threats, where the attack surface extends well beyond the server to the final binary product.

This structural vulnerability is why software supply chain attacks represent an asymmetrical threat that neutralizes traditional security paradigms. Because malicious code is imported through verified, trusted channels, it operates with the same high-level privileges as the legitimate application, bypassing firewalls and endpoint detection. Supply chain breaches are uniquely difficult to detect, as reports from IBM [5] highlight that these compromises take an average of 267 days to identify and contain, significantly increasing remediation costs. Consequently, Cybersecurity Ventures projects that the global cost of software supply chain attacks surpassed $60 billion in 2025 and is projected to reach $138 billion by 2031 [3].

A significant facet of this crisis is the prevalence of Zombie Packages, dependencies that remain technically functional but have been abandoned by their maintainers. Unlike traditional vulnerable packages, Zombie Packages may not have publicly disclosed CVEs while still representing substantial long-term security risk. Indicators such as prolonged inactivity, insufficient maintainer diversity, unresolved issue accumulation, or explicit package deprecation suggest that future vulnerabilities may remain permanently unpatched. Thus, these dependencies create latent attack surfaces that conventional CVE-centric scanners frequently fail to identify. To address this limitation, the primary objective of this project is to develop an automated audit framework capable of quantifying dependency sustainability through repository vitality metrics and heuristic risk analysis.

2. Related Work

Software supply chain security has become a significant area of inquiry within both the industrial and academic spheres, primarily required by the pervasive integration of third-party dependencies in contemporary software systems. Quantitative analysis from Sonatype [13], BlackDuck [1], and JFrog [8] substantiates this observation by demonstrating that modern application architectures are increasingly structured around extensive, multi-layered open-source dependency trees. The widespread adoption of this first-party external architecture strategy creates a critical security shift, as the integrity of the application increasingly relies on the ongoing maintenance and reliability of external code maintainers.

This structural dependence exposes development environments to different threat vectors, including maintenance decay, malicious package injection, and dependency confusion [14]. In response to these vulnerabilities, frameworks such as OpenSSF GuardDog [9] have been introduced to automate threat detection. Despite the emergence of such tools, the prevailing industry approach remains predominantly reactive, as it focuses primarily on the identification of known Common Vulnerabilities and Exposures (CVEs). Research by IEEE [7] and BoostSecurity [2] indicates that this vulnerability-centric strategy is insufficient for ensuring long-term repository integrity.

A critical challenge in maintaining repository integrity involves the assessment of maintenance vitality, where vitality metrics include commit frequency, community responsiveness, and the project bus factor. If a package contains no documented security flaws, it is frequently categorized as secure, despite potential abandonment by its maintainers. These neglected components, defined as Zombie Packages, represent a form of technical debt that standard scanning tools fail to evaluate. Unlike malicious code injection, which is identified through attack signatures, the risk associated with Zombie Packages is accumulative, requiring a broader assessment of repository health. Within the scope of this project, Zombie Packages are treated as dependencies exhibiting advanced maintenance decay capable of introducing long-term software supply chain exposure independently of currently disclosed vulnerabilities.

This evolution in risk management has led the industry to adopt multidimensional assessment models designed to overcome the constraints of reactive scanning. Implementing heuristic models to quantify dependency obsolescence and community activity provides the data-driven insights necessary to evaluate dependency sustainability. Consequently, identifying high-risk components prior to integration into production environments facilitates a more robust approach to maintaining software integrity.

This multidimensional approach is supported by recent research which identifies structural indicators, such as supply chain smells, as better predictors of risk than traditional vulnerability databases [11]. Industry data reinforces this, indicating that security debt in software projects is largely caused by neglected maintenance rather than isolated coding errors [6]. Further investigation into how vulnerabilities are reported shows that effective security depends heavily on the sustained maintenance and organizational health of a repository [10]. Although there is growing agreement on the need for these sustainability metrics, current tools lack a unified framework to integrate them into automated development pipelines. Thus, this gap between theoretical models and practical implementation serves as the primary motivation for our project.

3. Requirements

The requirements defined in this section operationalize the theoretical models previously discussed by translating the identified gaps in current security practices into specific technical objectives.

3.1. Functional Requirements

The functional requirements specify the operations the system must execute to identify "Zombie Packages" and evaluate repository health:

- **FR1 - Dependency Extraction:** The system shall parse package manifest files, including `package.json`, `requirements.txt`, and `go.mod`, to identify and extract external software dependencies from the target repository.
- **FR2 - Repository Resolution:** The framework shall correctly resolve platform-specific package aliases, package shorthands, and Go Vanity URLs to their corresponding GitHub source repositories for accurate metadata collection.
- **FR3 - Vulnerability Intelligence Integration:** The system shall integrate with the Google Open Source Vulnerability (OSV) API to identify publicly disclosed Common Vulnerabilities and Exposures (CVEs) associated with each detected dependency.
- **FR4 - Repository Vitality Analysis:** The framework shall interact with the GitHub REST API to collect repository maintenance metrics, including commit activity, contributor count, and unresolved issue statistics.
- **FR5 - Risk Score Computation:** The tool shall implement a mathematical heuristic model capable of calculating a dependency risk score within the range of 0 to 100 based on vulnerability exposure, inactivity, and maintenance neglect indicators.
- **FR6 - Zombie Package Classification:** Dependencies exceeding the defined risk threshold or containing explicit deprecation indicators shall be formally classified as *Zombie Packages*.
- **FR7 - Repository Security Classification:** The framework shall assign a repository-level security classification (*Safe*, *Warning*, or *Dangerous*) based on the highest dependency risk score identified within the dependency tree.
- **FR8 - Real-Time Audit Visualization:** The system shall provide a real-time terminal dashboard displaying dependency analysis summaries and detailed heuristic metrics for each analyzed package.
- **FR9 - Report Generation:** The framework shall generate structured JSON audit reports containing complete dependency analysis results for integration with external CI/CD pipelines and security automation workflows.

3.2. Non-Functional Requirements

The following non-functional requirements define the technical constraints and quality attributes of the system:

- **NFR1 - Performance Efficiency:** The system shall execute external API queries concurrently in order to reduce the total dependency audit execution time and improve overall analysis performance.
- **NFR2 - Scalability:** The framework shall support the use of authentication tokens to manage API rate limits and ensure stable operation during large-scale dependency analysis processes.
- **NFR3 - Maintainability:** The system architecture shall decouple dependency analysis logic from visualization and presentation layers to facilitate independent maintenance, debugging, and future extensibility.
- **NFR4 - Portability:** The application shall remain compatible with WSGI-compliant server standards to ensure deployability across multiple production environments and Linux-based infrastructures.
- **NFR5 - Interoperability:** The framework shall expose its core functionality through a REST API interface to support integration with external CI/CD pipelines, automated security workflows, and third-party platforms.

4. Methodology

The design of this framework prioritizes a modular architecture capable of automating the detection of software supply chain risks. Therefore, this chapter describes the system's structural components and the coordination between internal logic and external data providers, concluding with a case analysis that demonstrates the practical necessity of the implemented solution.

4.1. System Architecture

At its most fundamental level, the system acts as an intermediary between the user and global security databases. The interaction between the auditor, the internal tool, and the external data sources, specifically the GitHub platform for project activity and the Open Source Vulnerabilities database, is illustrated in Figure 1.

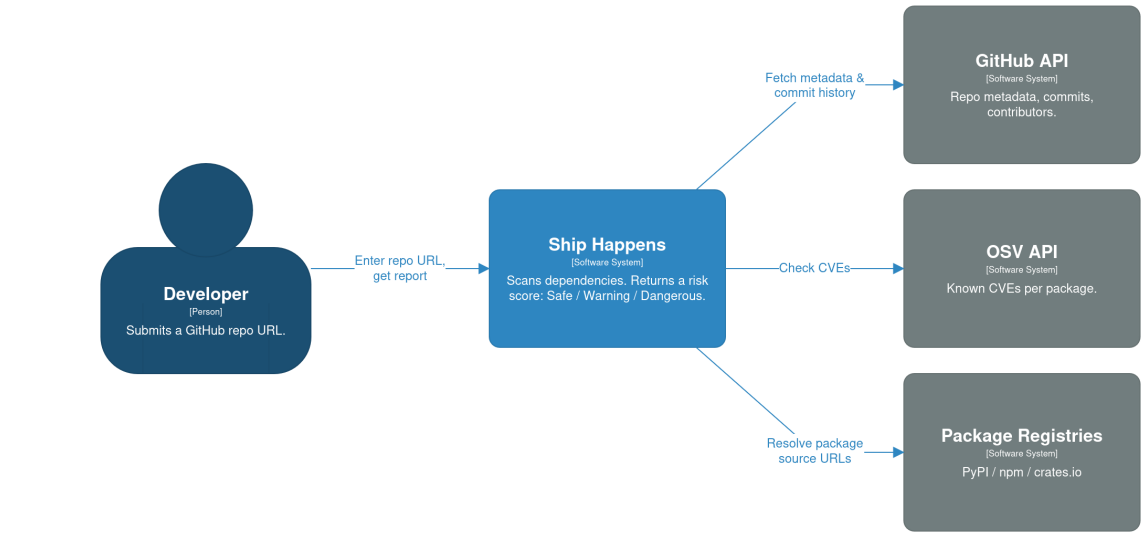


Figure 1: Project Overview Diagram

To ensure the tool is accessible both as a desktop application and through a web browser, the design is partitioned into functional units. This structural division is further detailed in Figure 2, which depicts how the visual dashboard interacts with the underlying processing services.

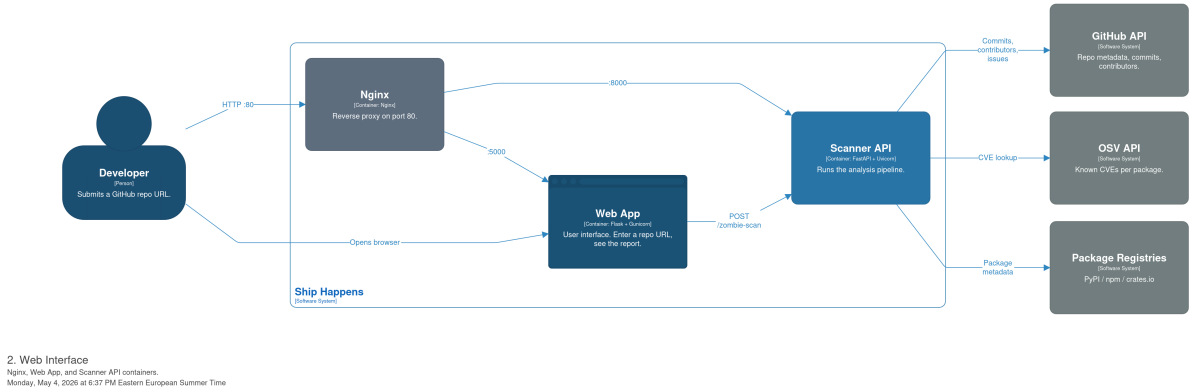


Figure 2: Web Interface Diagram

While the web interface manages user interaction, the technical execution of each audit is governed by a dedicated communication bridge. As shown in Figure 3, this component orchestrates the internal lifecycle of a scan, from the initial reading of project manifest files to the simultaneous collection of security data.

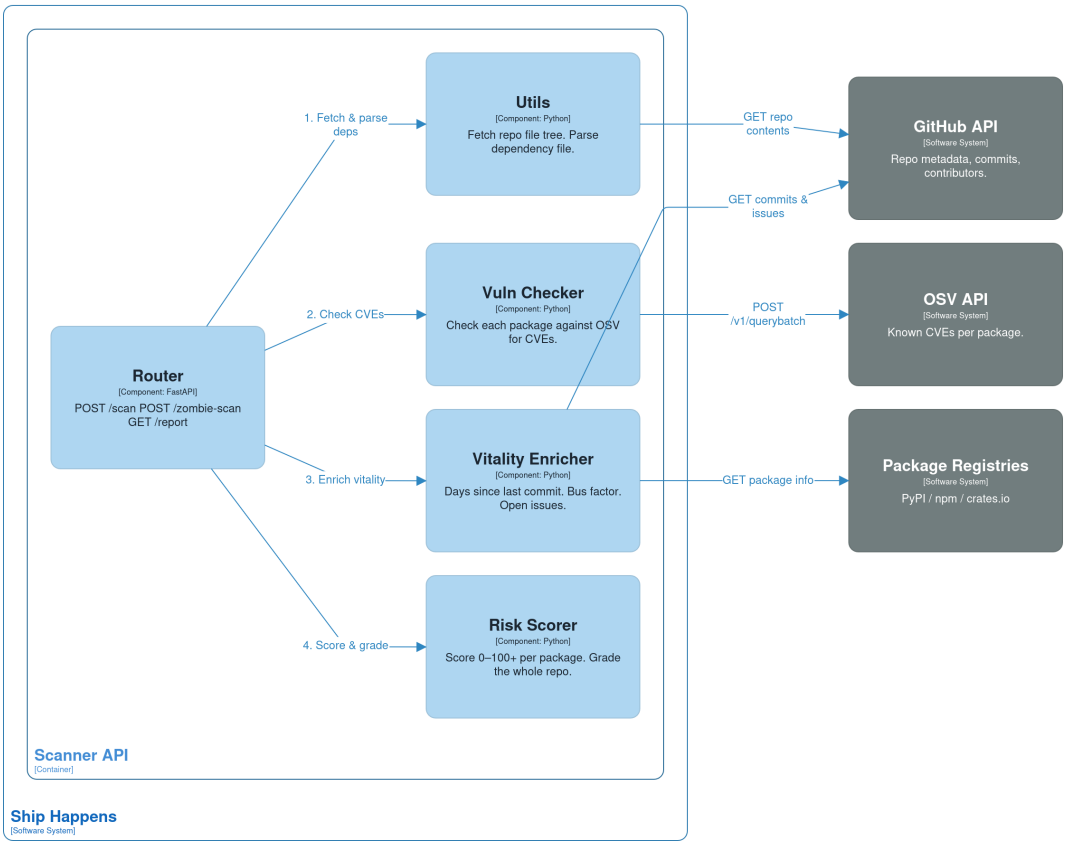


Figure 3: API Flow Diagram

The physical organization of the project reflects this logical modularity through a cohesive directory structure. The core analysis engine, located within the `dependency_reader` folder, implements the logic required to apply the security heuristic, while the `webapp` folder manages the visual presentation. Operational stability is further maintained through configuration scripts and environment files, such as `setup.sh` and `requirements.txt`, which ensure that the tool performs consistently across different computer systems.

Following the architectural design, the core analytical capability of the framework is implemented through a heuristic-based scoring model designed to quantify dependency security and maintenance health.

4.2. Risk Scoring Heuristic

To simplify the interpretation of dependency health and security posture, the proposed framework computes a quantitative *Risk Score* for every dependency on a bounded scale from 0 to 100, where:

$0 \rightarrow$ Completely Safe

$100 \rightarrow$ Highly Dangerous or Zombie Dependency

The heuristic is designed as a weighted aggregation of four independent security and maintenance indicators:

- Known Vulnerabilities (CVEs)
- Repository Inactivity
- Bus Factor
- Issue Neglect

The final dependency risk score is defined as:

$$R_d = C_{cve} + C_{inact} + C_{bus} + C_{issues}$$

where:

- R_d represents the final dependency risk score
- C_{cve} represents the vulnerability contribution
- C_{inact} represents the inactivity contribution
- C_{bus} represents the bus factor contribution
- C_{issues} represents the issue neglect contribution

The score is capped to the interval:

$$0 \leq R_d \leq 100$$

4.2.1. CVE Contribution

The vulnerability contribution is derived from the number of publicly disclosed vulnerabilities detected through the Open Source Vulnerability (OSV) database.

Each detected CVE contributes 10 points to the overall score:

$$C_{cve} = 10 \times N_{cve}$$

where:

- N_{cve} represents the total number of known vulnerabilities

The contribution is bounded to a maximum of 50 points:

$$C_{cve} \leq 50$$

4.2.2. Inactivity Contribution

The inactivity heuristic evaluates the maintenance freshness of a package based on the elapsed time since the last repository commit.

For every 6 months of inactivity, 10 points are added:

$$C_{inact} = 10 \times \left\lfloor \frac{M}{6} \right\rfloor$$

where:

- M represents the number of months since the last commit

This contribution is limited to 20 points:

$$C_{inact} \leq 20$$

4.2.3. Bus Factor Contribution

The bus factor heuristic evaluates the resiliency of project maintenance by considering the number of active maintainers.

Repositories maintained by one or fewer contributors receive the maximum penalty:

$$C_{bus} = \begin{cases} 15, & \text{if } N_{maint} \leq 1 \\ 0, & \text{otherwise} \end{cases}$$

where:

- N_{maint} represents the number of active maintainers

4.2.4. Issue Neglect Contribution

The issue neglect heuristic measures unresolved maintenance burden through the amount of open unresolved issues.

For every 20 open issues, 5 points are added:

$$C_{issues} = 5 \times \left\lfloor \frac{N_{issues}}{20} \right\rfloor$$

where:

- N_{issues} represents the total number of open issues

The contribution is capped at 15 points:

$$C_{issues} \leq 15$$

4.2.5. Zombie Dependency Classification

A dependency is classified as a *Zombie Dependency* if at least one of the following conditions is satisfied:

$$R_d \geq 100$$

or

$$D_{flag} = 1$$

where:

- D_{flag} represents an explicit deprecation indicator provided by the package registry

Zombie dependencies are considered high-risk due to severe abandonment, lack of maintenance, or critical unresolved vulnerabilities.

4.3. Overall Repository Security Heuristic

The overall repository security score is determined by the highest dependency risk score identified within the dependency tree.

Let:

$$R_{repo} = \max(R_{d_1}, R_{d_2}, \ldots, R_{d_n})$$

where:

- R_{repo} represents the repository risk score
- R_{d_i} represents the risk score of dependency i

This design assumes that a software repository is only as secure as its weakest dependency. This approach prioritizes conservative security assessment, since a single severely compromised dependency may be sufficient to compromise the integrity of the entire software supply chain.

4.4. Repository Security Classification

Based on the final repository risk score, repositories are categorized into three security tiers:

Range	Classification	Description
0 – 50	SAFE	Strong maintenance activity and healthy security posture.
51 – 70	WARNING	Presence of aging dependencies or moderate unresolved vulnerabilities.
71 – 100	DANGEROUS	High probability of supply chain compromise due to abandoned dependencies or severe vulnerabilities.

Table 1: Repository Security Classification

5. Case Analysis

To validate the proposed framework, we evaluated several real-world GitHub repositories: Requests, Django, and Ansible, as well as the dependency ecosystem involved in the recent Mistral AI supply chain incident. The experimental results, summarized in Table 2, confirm that repository popularity and a lack of known CVEs (Common Vulnerabilities and Exposures) are often insufficient indicators of true dependency security.

The Requests repository received a risk score of 65, placing it in the *WARNING* category. While the core project remains actively maintained, underlying dependencies like `urllib3` drove up the risk score due to a combination of 17 open CVEs and over 200 unresolved issues. Ansible yielded the highest risk profile in our evaluation, scoring 83 and receiving a *DANGEROUS* classification. In this case, dependencies such as `jinja2` and `PyYAML` exhibited a compounding risk factor: documented vulnerabilities paired with prolonged maintainer inactivity and a mounting backlog of open issues.

Conversely, Django presented the healthiest ecosystem, securing a *SAFE* classification with a score of 24. Even so, the framework successfully flagged latent maintenance risks within its dependency tree; for instance, `grunt-cli` accumulated a high heuristic score despite having zero disclosed CVEs, driven entirely by a complete absence of commits for over 600 days. This outcome validates a primary objective of our framework: uncovering hidden supply chain risks before they manifest as public vulnerabilities.

The Mistral AI case study further underscores the value of merging vulnerability intelligence with repository health metrics. The analyzed dependency tree generated an overall risk score of 65. Key dependencies like `pillow` and `requests` each reached individual risk scores of 65, burdened by unresolved issues and known vulnerabilities. Notably, these results demonstrate that our heuristic could identify severe supply chain vulnerabilities within the ecosystem well before the broader operational fallout of the incident occurred.

5.1. Heuristic Refinement and "Zombie Package" Detection

A critical finding during these experiments involved the practical behavior of the `vitality_enricher.py` module. Initially, the "Zombie Package" heuristic was designed to flag packages that were simultaneously actively used, neglected, and managed by a single author. However, our integration with the GitHub API revealed that a `maintainer_count` ≤ 1 almost never triggers in a real-world environment.

Because the GitHub API counts all-time historical contributors, even a project abandoned five years ago typically displays 10–20+ contributors from its period of peak activity. This demonstrates an inherent design tension in naive threshold-based detection: the three states required for a theoretical zombie package are often mutually exclusive in practice. Consequently, the framework was tuned to a pragmatic compromise of ≤ 10 maintainers and > 5 open issues. This adjustment allows the model to maintain a proactive security posture by identifying "neglected" projects that still appear "populous" due to historical metadata.

Repository	**Score**	**Classification**	**Highest Risk Dependency**
Requests	65	WARNING	`urllib3`
Django	24	SAFE	`grunt-cli`
Ansible	83	DANGEROUS	`jinja2`
Mistral AI Ecosystem	65	WARNING	`pillow`

Table 2: Repository Risk Evaluation Summary

Taken together, these experiments show that analyzing maintenance activity, contributor diversity, inactivity intervals, and issue backlogs yields a far more robust security evaluation than relying strictly on CVE data. Crucially, these scores serve as probabilistic indicators of ecosystem health rather than binary guarantees of architectural compromise.

6. Conclusions

This project demonstrates that a more effective way to evaluate software supply chain security is by combining traditional vulnerability tracking with real-time repository health analysis. By shifting away from standard, CVE-centric models, our framework incorporates metrics like maintainer activity, contributor diversity, and issue neglect to expose hidden risks within abandoned or poorly managed dependencies.

Our findings reveal that even highly popular, actively maintained top-level repositories can quietly inherit severe supply chain risks through transitive dependencies that suffer from neglect, long-term inactivity, or structural vulnerability. Moreover, the framework's ability to flag high-risk dependencies that carry zero official CVEs underscores the practical necessity of a proactive, maintenance-first assessment strategy.

The resulting Risk Score Heuristic effectively categorizes repositories by real-world threat levels and introduces the concept of *Zombie Packages* to systematically isolate abandoned software. Furthermore, the development of our web-based platform proves that complex, automated dependency health checks can be successfully operationalized into lightweight, accessible security tools.

Beyond identification, the framework guides defensive remediation. It provides actionable insights for updating vulnerable components, replacing dead libraries with maintained alternatives, trimming unnecessary dependencies, or engineering temporary internal patches when upstream maintainers fail to respond.

While heuristic analysis cannot offer flawless security guarantees, this approach marks a clear step toward identifying supply chain threats earlier in the lifecycle. By looking past historical vulnerability disclosures and focusing on the long-term sustainability and operational health of an ecosystem, security teams can proactively manage risk before a compromise occurs.

References

[1] BlackDuck. Open source security and risk analysis report 2025. Technical report, BlackDuck, 2025. Accessed: 2026-04-04.

[2] BoostSecurity. Don't go with the flaw: Supply chain risks. *BoostSecurity Blog*, 2025. Accessed: 2026-04-04.

[3] Cybercrime Magazine. Global costs of software supply chain attacks on the rise. *Cybersecurity Ventures*, 2025. Accessed: 2026-04-01.

[4] Cycode. Shedding the lite: Unfolding the dramatic turn of events with the litellm compromise. *Cycode Blog*, 2026. Accessed: 2026-04-01.

[5] DeepStrike. Supply chain attack statistics 2025: Costs, cases, defenses. *DeepStrike Research*, 2025. Accessed: 2026-04-01.

[6] Help Net Security. Open-source security debt grows across commercial software. *Help Net Security*, 2026. Accessed: 2026-04-17.

[7] IEEE. Supply chain security assessment in modern ci/cd. *IEEE Xplore*, 2023. Accessed: 2026-04-04.

[8] JFrog. State of software supply chain security 2025. Technical report, JFrog, 2025. Accessed: 2026-04-04.

[9] OpenSSF. Guarddog: Strengthening open source security. *OpenSSF Blog*, 2025. Accessed: 2026-04-04.

[10] A. Ouni et al. On categorizing open source software security vulnerability reporting mechanisms on github. *ResearchGate*, 2026. Accessed: 2026-04-17.

[11] A. Silva and T. Mens. Software supply chain smells: Lightweight analysis for secure dependency management. *arXiv preprint arXiv:2603.24282*, 2026. Accessed: 2026-04-17.

[12] Snyk. How a poisoned security scanner became the key to backdooring litellm. *Snyk Security*, 2026. Accessed: 2026-04-01.

[13] Sonatype. State of the software supply chain 2025. Technical report, Sonatype, 2025. Accessed: 2026-04-04.

[14] Various Authors. A survey of supply chain security in open source. arXiv:2406.10109, 2024. Accessed: 2026-04-04.

[15] Verizon. 2025 data breach investigations report (dbir). *Verizon Business*, 2025. Accessed: 2026-04-01.